

TNT — 한국어 의역본

TNT — 한국어의역본

이 문서는 arXiv:2511.07343 「TNT: Improving Chunkwise Training for Test-Time Memorization」 (Li, Behrouz et al. (USC / Google Research))의 한국어 의역본입니다. 스타디용이며 공식 번역이 아닙니다. 수식 · 수치 · 그림은 원문 기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해 문장은 의역했습니다.

초록 (Abstract)

Titans, TTT처럼 깊은 **test-time memorization 모듈 (deep test-time memorization module)**을 갖춘 recurrent neural network(RNN)는, Transformer와는 계보가 다르면서도 시퀀스 길이에 선형으로 확장되는 유망한 패러다임이다. 이 표현력 높은 모델들은 아직 최고 성능의 Transformer를 따라잡지는 못했지만, 그 잠재력은 지금까지 거의 활용되지 못했다. 이유는 훈련이 지나치게 느리고 하드웨어 활용률이 낮기 때문이다. 기존의 병렬화 기법들은 **chunksize(체크 크기)** 하이퍼파라미터가 지배하는 본질적 상충 관계에 갇혀 있다. 큰 체크는 속도를 끌어올리지만 성능을 떨어뜨리고, 그 결과 어쩔 수 없이 고정된 차선(次善)의 타협값을 골라야만 한다.

이 문제를 풀기 위해 우리는 TNT를 제안한다. TNT는 훈련 효율과 추론 성능을 **두 단계 (two-stage)** 과정을 통해 분리 (decouple) 하는 새로운 훈련 패러다임이다. 1 단계는 효율에 초점을 둔 사전학습 단계로, **계층적 메모리 (hierarchical memory)**를 사용한다. 여기서 global 모듈은 장거리 문맥을 담당하며 하드웨어에 친화적인 큰 체크를 처리하고, 여러 개의 병렬 local 모듈은 세밀한 정보를 다룬다. 핵심은, local 메모리 상태를 **주기적으로 리셋 (periodic reset)** 함으로써 순차적 의존성을 끊고 대규모 **context 병렬화 (context parallelization)**를 가능하게 한다는 점이다. 2 단계는 짧은 fine-tuning 단계로, **local 메모리 모듈만** 더 작고 고해상도인 체크 크기에 맞춰 조정하여, 최소한의 오버헤드로 정확도를 극대화한다.

Titans와 TTT 모델에 적용해 평가한 결과, TNT는 훈련 속도에서 상당한 가속—가장 정확한 baseline 설정 대비 최대 17배 빠름—을 달성하면서 **동시에** 모델 정확도까지 향상시킨다. 이 개선은 중요한 확장성 장벽을 제거하여, 표현력 있는 RNN을 개발할 실용적 토대를 마련하고, Transformer와의 성능 격차를 좁히는 후속 연구를 촉진한다.

1 서론 (Introduction)

긴 시퀀스를 모델링하려는 수요는 표준 softmax attention (Vaswani et al. 2017)의 근본적 한계를 드러낸다. 즉, **이차 (quadratic) 복잡도**가 확장의 병목이 된다. 이 때문에 더 효율적인 아키텍처를 향한 광범위한 연구가 촉발되었다. 그중에서도 특히 강력한 접근이 **test-time memorization** (Sun, Li, et al. 2024)에 뿌리를 둔 방법이다. 이 원리를 활용하는 아키텍처를 우리는 **deep memory module**이라 부르며, 이들은 깊고 온라인으로 적응되는 서브네트워크를 사용해 그 파라미터를 빠르게 갱신함으로써 in-context 정보를 인코딩한다. 대표적 예로 Titans (Behrouz, Zhong, et al. 2025)와 Atlas (Behrouz, Li, et al. 2025)가 있다. 이는 **linear memory module** (Yang, Wang, Shen 등; Dao and Gu 2024; Sun, Dong 등)과 뚜렷이 대비된다. 후자는 효율적이지만 행렬 값의 은닉 상태와 선형 상태 전이 (linear state transition)에 제약된다. deep memory module은 표현력 높은 비선형 목적 함수와 갱신 규칙을 사용함으로써 이론적으로 그런 한계를 넘어설 수 있다. 이 방법들이 아직 Transformer의 SOTA 성능에 도달하지는 못했지만, 훈련 병목만 해소된다면 효율적 시퀀스 모델링의 유망한 패러다임이 될 수 있다.

표현력의 이점에도 불구하고, deep memory module은 선형 계열이 누리는 효율적 훈련 알고리즘을 갖지 못해 하드웨어 활용률이 낮다. linear memory module은 하드웨어 효율적 병렬화를 쓸 수 있는 반면, deep memory module은 비선형 recurrence (예: 체크 사이의 LayerNorm)와 깊은 구조의 복잡성에서 오는 어려움에 직면한다. 실제로 이 어려움 때문에 이들의 훈련은 **작은 데이터 조각 위에서 더 잦은 온라인 갱신**에 갇히게 되고, 훈련 처리량 (throughput)이 나빠진다. 그 결

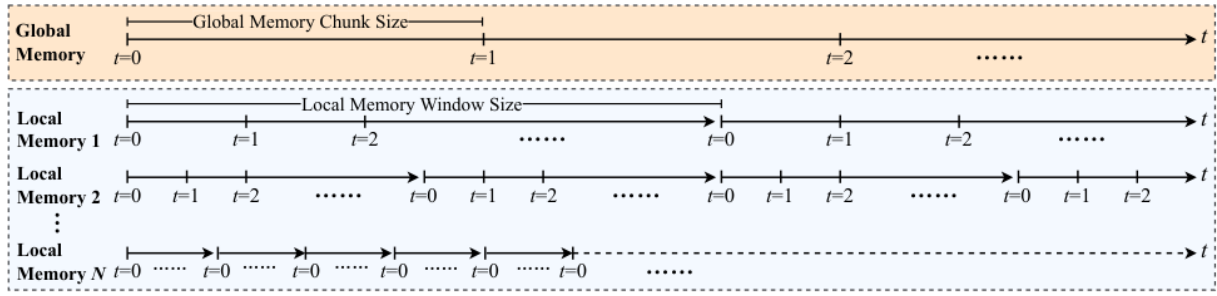


그림 1: TNT 메모리 계층 구조를 나타낸 기본 다이어그램. 각행에서, 같은 t 값을 갖는 갱신들은 동시에 (병렬로) 실행된다. $t=0$ 은 메모리의 초기화 시점이다. (출처: 원저자, arXiv:2511.07343 Fig.1)

과 본질적 긴장이 발생하는데, 이 모델들은 보통 메모리 층의 표현력과 훈련 효율을 저울질하기 위해 **고정된 작은 청크 크기 (예: 16 – 64 토큰)**에 의존한다. 결국 in-context 학습 능력과 계산 성능 사이의 이 트레이드오프가, 이 모델들을 실제로 긴 시퀀스에 적용하는 것을 가로막는 결정적 병목이 되어 왔다. 이 근본적 긴장을 해소하는 것이 본 연구의 주된 목표다.

최근 연구들이 이 문제를 완화하려 시도했다. Zhang, Bi, et al.(2025) 은 큰 청크에 local attention 을 결합해 병렬성을 높인다. 그러나 이는 비효율을 해결하는 대신 **우회** 하는 것이고, memory 와 attention 을 섞어 분석을 복잡하게 만들며, 추론 시 작은 청크 (이상적으로는 1) 가 필요하다는 점을 간과한다. 같은 시기 Guo et al.(2025) 은 계층적 메모리 시스템을 제안했지만, 이는 linear memory module 에 국한되며 단기 메모리를 지원하지 않는다.

이 긴장을 해소하기 위해 우리는 deep memory module 을 위한 새로운 훈련 패러다임 **TNT** 를 제안한다. 우리의 핵심 통찰은, 모델의 서로 다른 구성 요소가 서로 다른 훈련 단계에서 서로 다른 **granularity (입도)** 로 정보를 처리해야 한다는 것이다. TNT 는 처리량을 극대화하도록 설계된 효율 중심의 사전 학습 단계에서 출발한다. 이는 계층적 메모리 시스템으로 달성된다. global 메모리 모듈은 하드웨어에 친화적인 큰 청크로 동작해 장거리 문맥을 포착하고, 여러 local 메모리 모듈은 세밀한 정보를 병렬로 처리한다. 결정적으로 우리는 local 메모리 상태에 대해 **주기적 리셋 메커니즘** 을 도입한다. 이는 비선형 RNN 에도 내재하는 (예: 단계 사이에 정규화가 들어간 경우) 순차적 의존성을 끊어, 대규모 context 병렬화를 가능하게 한다. 시퀀스 길이 방향으로 비선형 recurrence 를 효율적으로 병렬화하는 것은 오랜 난제였고, Transformer 와 (parallel scan 이 적용되는) 특수한 linear RNN 을 제외하면 대체로 미해결 상태였기에, 이는 핵심 혁신이다. 이어지는 성능 중심의 fine-tuning 단계는 최적 추론을 위해 모델을 조정한다. 이 단계에서는 **local 메모리 모듈만** 더 작은 청크 크기를 쓰도록 조정되어, 최소한의 추가 계산으로 고해상도 정확도를 얻는다. 이 2 단계 접근은 훈련 효율과 추론 성능을 효과적으로 분리하여, 훈련 확장성을 크게 개선하는 동시에 기존 아키텍처의 핵심 한계를 해결한다. 나아가 local 메모리 시스템 자체가 계층적일 수 있어, 서로 다른 해상도로 동작하는 여러 모듈을 둘 수 있다. 이 **다중해상도 (multi-resolution)** 접근은 단일 고정 청크보다 복잡한 다중 스케일의 시간적 동역학을 더 효과적으로 포착하게 해 준다.

TNT 는 특정 아키텍처가 아니라, **어떤 deep memory module 에도 적용 가능한 일반적 훈련 패러다임** 이다. 훈련 처리량과 추론 정확도를 분리함으로써, 선행 연구를 제약하던 근본적 긴장을 해소한다. 이는 작은 청크를 위한 하드웨어별 최적화에 대한 의존을 없애고, 아키텍처 설계 공간의 유연한 탐색을 가능하게 한다. 우리는 이 패러다임이 softmax attention 을 대체하려는 새로운 연구 방향을 열 것이라 믿는다.

주요 기여를 요약하면 다음과 같다.

- deep memory module 의 확장성과 성능을 제한하는 세 가지 근본 도전을 규명한다: 1) 메모리 compression 과 retrieval 사이의 **domain mismatch**, 2) 메모리 성능과 계산 효율 사이의 트레이드오프, 3) 훈련과 추론 사이의 **chunksize mismatch** (3 절).
- compression – retrieval 간 domain mismatch 를 해소하는 효율적 메커니즘인 **Q-K Projection** 을 도입한다 (4.1.2 절).
- 주기적 상태 리셋을 갖춘 새로운 계층적 메모리 아키텍처를 도입하여, 비선형 deep memory module 에 대한 **context 병렬화** 를 가능하게 한다 (4.1 절).

- 훈련-추론 chunksize mismatch 를 해결하는 효율적 fine-tuning 메커니즘을 도입한다 (4.2 절).
- 이 모두를 결합해, 효율적 사전학습과 고해상도 fine-tuning 을 합쳐 훈련효율과 추론 성능을 분리하는 일반적 2 단계 훈련 패러다임 TNT 를 제시한다 (Figure 1, Figure 3, 4 절).
- Titans 아키텍처 위에서 TNT 를 검증하여, 정확도를 향상시키면서도 최대 **17.37 배** 훈련 속도를 달성하며, 표현력 있는 RNN 의 실용성을 크게 끌어올린다 (5 절).

문제 정의와 표기. 우리는 파라미터 $\theta \in \mathbb{R}^{d_m}$ 를 갖는 신경망을 다음 토큰예측 (next-token prediction) 을 수행하도록 훈련하는 것을 목표로 한다. 시퀀스 $x = (x_1, \dots, x_L)$ 가 주어지면, 모델의 목표는 각 토큰 x_t 를 그 앞선 토큰들 $(x_1, \dots, x_{t-1})$ 의 문맥을 사용해 예측하는 것이다. attention 정식화를 따라, 각 토큰 x_t 는 d 차원 벡터로 표현된다. 각 입력 토큰 x_t 는 query, key, value 벡터 $q_t, k_t, v_t \in \mathbb{R}^d$ 로 사영된다. 이후 chunkwise 연산의 표기를 위해, 함수

$$\xi(i, j) := i - (i \bmod j)$$

를 정의한다. 이는 청크 크기가 j 일 때 인덱스 i 를 포함하는 청크의 시작 인덱스를 찾아 준다.

2 배경 (Preliminary)

이 절은 배경 지식을 정리한다. 확장된 관련 연구는 부록 A 에 있다.

2.1 Test-Time Memorization 을 통한 Deep Memory Module

강력한 시퀀스 모델링 패러다임 중 하나는 **Test-Time Memorization** (Sun, Li, et al. 2024) 이다. 이는 빠르게 적응 가능한 보조신경망을 결합해 모델을 강화한다. 훈련 중에만 갱신되는 주 (主) 모델파라미터, 즉 **slow weights** θ 와 달리, 이 접근은 **fast weights** (Schlag et al. 2021) 를 도입한다. W 로 표기되는 이 fast weights 는 서브네트워크 $f(W, \cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ 를 파라미터화하며, 들어오는 토큰에 기반해 **훈련과 추론 양쪽에서 온라인으로** 갱신되어 문맥 정보를 동적으로 저장한다. 이 모듈들은 아직 Transformer 대비 SOTA 에 이르지 못했지만 (Arora et al. 2024; Behrouz, Li, et al. 2025), 격차를 좁히는 데 필요한 폭넓은 실험을 가능하게 하려면 훈련 효율 개선이 필수적이다.

본 연구에서 우리는 이와 유사·관련된 원리인 **deep memory module** (Irie et al. 2021; Sun, Li 등; Behrouz, Zhong 등; Behrouz, Li 등; Behrouz, Razaviyayn 등 2025a) 에 초점을 둔다. 선형 상태 전이로 특징지어지는 linear memory module (Sun, Dong 등; Yang, Wang, Zhang 등; Dao and Gu 2024; Karami and Mirrokni 2025; Hu, Pan 등 2025) 과 달리, deep memory module 은 비선형 recurrence 규칙과 복잡한 메모리 구조를 사용한다.

deep memory module 의 핵심 메커니즘은 각 입력 토큰마다 이루어지는 두 개의 순차적 연산 — 1. **Memory Compression**, 2. **Memory Retrieval** — 으로 요약할 수 있다. 형식적으로는 다음과 같다.

$$\text{Memory Compression: } W_t \leftarrow W_{t-1} - \eta_t \nabla_W \mathcal{L}(f(W_{t-1}, k_t), v_t) \quad (1)$$

$$\text{Memory Retrieval: } o_t = f(W_t, q_t) \quad (2)$$

Memory Compression에서는 fast weights W 가 자기지도 손실 $\mathcal{L}(\cdot, \cdot)$ (예: MSE)과 학습된 학습률 η_t 에 이끌려 gradient descent로 갱신된다. 이 식은 변환된 key $f(W_{t-1}, k_t)$ 를 그 value v_t 와 연결 (associate)시켜, 정보를 고정 크기의 신경 메모리에 압축한다는 뜻이다. Memory Retrieval에서는 갱신된 W_t 가 query q_t 를 처리해 출력 o_t 를 만든다. 이 두 연산이 각 토큰마다 반복적으로 수행된다.

2.2 Chunkwise 병렬 훈련 (Chunkwise Parallel Training)

식 1-2에 있는 순차적 의존성 (W_t 가 W_{t-1} 에 의존)은 시퀀스길이 방향의 병렬화를 막는다. 이를 해결하기 위해 deep memory module은 하드웨어 효율적 훈련을 가능케 하는 **chunkwise parallel training** (Hua et al. 2022; Sun, Dong, et al. 2023)을 채택한다.

핵심 원리는, 입력 시퀀스를 크기 C 의 서로 겹치지 않는 청크들로 나누는 것이다. 각 청크 안에서는, 모든 토큰의 손실 gradient를 그 청크 시작 지점의 fast-weight 상태에 대한 근사로 계산한다. 이 정식화는 gradient 계산에서의 토큰-대-토큰 순차 의존성을 끊어, 청크 내 모든 토큰의 갱신을 병렬로 계산할 수 있게 한다. 시점 t 의 토큰에 대한 형식적 연산은 다음과 같다.

$$\text{Chunkwise Memory Compression: } W_t \leftarrow W_{\xi(t,C)} - \sum_{\tau=\xi(t,C)}^t \eta_\tau \nabla_W \mathcal{L}(f(W_{\xi(t,C)}, k_\tau), v_\tau) \quad (3)$$

$$\text{Chunkwise Memory Retrieval: } o_t = f(W_t, q_t) \quad (4)$$

여기서 $W_{\xi(t,C)}$ 는 토큰 t 를 포함하는 청크 시작 지점의 fast weights 상태다 ($\xi(\cdot, \cdot)$ 의 정의는 1절 말미 참조). W_t 를 얻는 갱신은 여전히 청크 내 앞선 토큰들에 의존하지만, gradient들의 합은 병렬 연산 (예: 누적 합, cumulative summation)으로 효율적으로 구현할 수 있어 훈련 중 하드웨어 활용률을 크게 높인다. 다만 순차 의존성 하나는 남는다. n 번째 청크의 최종 fast-weight 상태 W_{nC} 가 $(n+1)$ 번째 청크의 초기 상태로 쓰인다는 점이다.

3 Deep Memory Module의 도전 과제 (Challenges)

chunkwise 병렬화는 deep memory module이 긴 시퀀스에서 훈련되도록 해주지만, 이 패러다임은 실용적 성능과 확장성을 제한하는 중대한 도전들을 낳는다. 이 절에서는 deep memory module의 세 가지 근본 도전을 정리한다.

Challenge 1: 효율적 훈련 구현의 부재. deep memory module의 일차적 도전은 훈련 과정의 비효율로, 이는 낮은 하드웨어 활용률로 이어진다. chunkwise 병렬화가 이론적으로는 sub-quadratic 확장을 가능케 하지만, 실제로는 훈련 처리량이 linear memory module에 크게 뒤진다. 이 차이는 모델 표현력과 계산 효율 사이의 근본적 긴장에서 비롯된다.

세밀한 학습 신호를 유지하려면 deep memory module은 작은 청크 크기 (예: 16-64 토큰) (Sun, Li, et al. 2024)를 요구하는데, 이는 가속기를 포화시키지 못해 훈련을 (compute-bound가 아니라) **memory-bound**로 만든다. linear memory module은 맞춤 커널 (예: SRAM 활용) (Sun, Dong 등; Gu et al. 2023; Qin et al. 2024; Yang 등)을 쓰지만, 이는 선형 상태 전이에 의존하므로 deep memory module의 크고 비선형인 상태와는 호환되지 않는다.

그 결과 deep memory module은 극도로 낮은 FLOPs 활용률을 겪는데, 흔히 하드웨어 최대 성능의 5-10% 미만에 머문다 (Zhang, Bi, et al. 2025). 이 심한 비효율은 사전 학습을 지나치게 느리고 비싸게 만들어, 이표현력 있는 모델들을 실제로 긴 시퀀스에 적용하는 것을 가로막는 중대한 병목이 된다.

Challenge 2: 메모리 Compression과 Retrieval 사이의 불일치. 메모리 서브네트워크가 훈련되는 방식과 사용되는 방식 사이에는 근본적 불일치가 존재한다. Memory Compression(식 1)

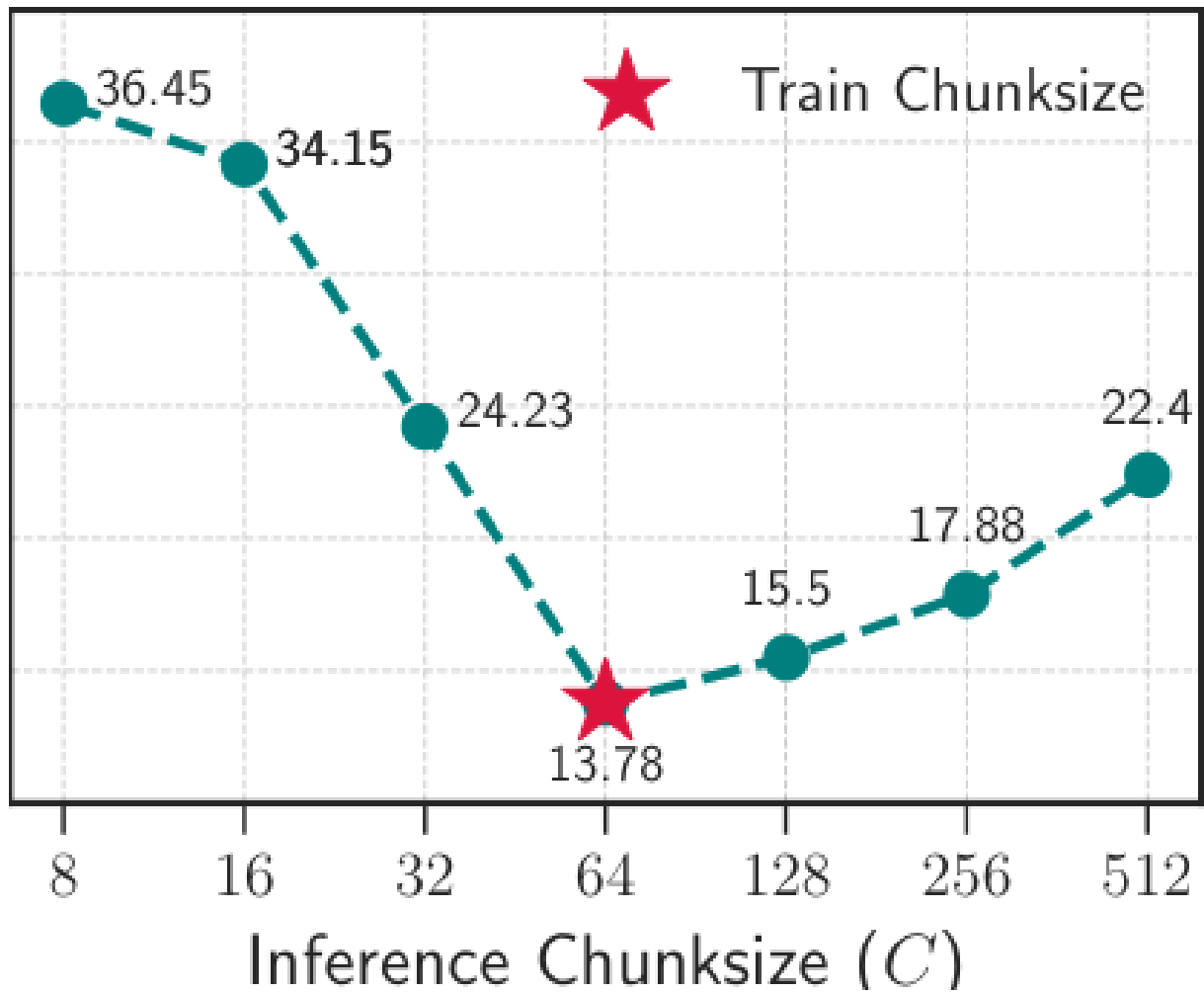


그림 2: $C=64$ 로 사전학습한 550M Titans 모델에서 추론 청크 크기에 대한 민감도. 추론 청크 크기가 훈련 때의 값과 일치할 때 성능이 최적이다. (출처: 원저자, arXiv:2511.07343 Fig.2)

동안 서브네트워크 $f(W, \cdot)$ 는 $\text{key}(k_t)$ 를 $\text{value}(v_t)$ 와 연결함으로써 **key 공간에서 value 공간으로의 사상** 을 학습하도록 최적화된다. 그런데 Memory Retrieval(식 2)에서는 key 가 아니라 **query 벡터** (q_t) 로 네트워크를 질의한다. 이 치환은 학습된 함수가 의도한 입력 domain 을 위반하여, 훈련 목적과 retrieval 과제 사이에 괴리를 만든다. 이 domain shift 는 학습된 사상의 무결성을 훼손하고 모델의 retrieval 성능을 제한할 수 있다. 이에 대한 실증적 검증은 5.4 절에 있다.

Challenge 3: 고정된 사전학습 Chunksize 에 대한 성능 민감도. 청크크기 하이퍼파라미터 C 는 훈련 처리량과 모델 표현력 사이의 트레이드오프를 지배한다. 현재 deep memory module 의 관행은 사전학습과 추론에 **동일한 고정 청크 크기** 를 쓰는 것이다. 그러나 우리는 추론시 성능이 이 사전학습 선택에 매우 민감함을 관찰한다. 예컨대 Figure 2 에서보듯, 청크 크기 64 로 사전학습한 모델은 바로 그 청크 크기로 평가할 때에만 최적의 perplexity 를 얻는다.

이 결과는 결정적인 train-test mismatch 를 드러내며, “추론 시 더 작은 청크가 더 ‘신선한’ 학습 신호로 세밀한 의존성을 포착해 더 나은 성능을 줄것” 이라는 직관과 모순된다. 오히려 모델은 훈련에서 본 특정 청크 해상도에 **과도하게 특화 (over-specialized)** 된다. 이 경직성은 중대한 한계다. 이상적으로는 크고 하드웨어 친화적인 청크 크기로 사전학습한 모델이, 추론 시 더 작고 정밀한 청크 크기에서 오히려 더 잘 동작할 만큼 유연해야 한다. 현재의 deep memory module 은 이 적응성을 달성하지 못한다.

4 TNT: Deep Memory 를 위한 개선된 훈련프레임워크

3 절에서 정리한 도전들을 해결하기 위해, 우리는 deep memory module 을 위한 개선된 훈련 패러다임 TNT 를 도입한다. 우리의 프레임워크는 훈련 효율과 추론 성능 사이의 내재적 긴장을 해소하도록 설계된 **2 단계과정—효율 중심 사전학습 단계 와 성능 중심 fine-tuning 단계—** 을 중심으로 구성된다.

1 단계는 전례 없는 병렬성을 가능케 하는 새로운 계층적 메모리 아키텍처를 도입하여 훈련 처리량을 극대화하고, 낮은 하드웨어 활용률과 불일치하는 메모리 목적 (Challenge 1–2) 을 직접 겨냥한다. 2 단계는 모델을 고해상도의 작은 청크 추론에 맞추는 효율적 fine-tuning 전략을 사용해, 사전학습 청크크기에 대한 민감도 (Challenge 3) 를 해결한다. 이 2 단계 접근은 훈련 효율과 추론 성능을 효과적으로 분리하여, 기존 deep memory 아키텍처의 핵심 한계를 극복한다.

4.1 TNT 1 단계: 효율 중심 사전학습

4.1.1 TNT Memory Compression: 계층적 메모리 순차적 상태 의존성은 context 병렬화 (시퀀스 shard 들을 여러 장치에 걸쳐 병렬로 처리하는 것) 를 막는다. 이를 가능하게 하기 위해, 우리는 모든 병렬 shard 가 자신의 local 메모리를 **동일한 학습된 상태** W_{init} 로 초기화하도록 제안한다. 이는 청크 간 (inter-chunk) 의존성을 끊어 대규모 병렬화를 허용한다. 하지만 이렇게 하면 local 메모리 모듈이 global 문맥을 잃게 된다. 이를 해결하기 위해 우리는 V 로 파라미터화된 **global 메모리 모듈** 을 도입하여, 샤딩된 local 메모리들과 병렬로 동작시킨다. global 메모리는 상대적으로 큰 청크 크기 (예: 2048 이상) 로 시퀀스를 처리하여, 높은 하드웨어 활용률을 유지하면서 장거리 의존성을 효율적으로 포착한다. 그 결과 local 메모리는 병렬 shard 안에서 세밀한 정보를 다루고 global 메모리는 전반적 문맥을 제공하는 계층 구조가 만들어진다.

이 계층 구조는 유연하다. 하나의 global 메모리 모듈과 N 개의 local 메모리 모듈로 모델을 설계할 수 있고, 각각 다른 해상도로 동작할 수 있다. 설명의 명료함을 위해 여기서는 가장 단순한 $N = 1$ 경우를 가정한다. TNT 의 일반화된 정식화는 부록 E 로 미룬다. 이제 메모리 compression 메커니즘을 형식적으로 정의한다.

TNT Memory Compression 규칙. 계층적 메모리는 다음과 같이 갱신된다.

Global Memory 갱신. global 메모리 상태 V 는 큰 청크 크기 C_G 로 입력 전체에 걸쳐 순차적으로 진화한다.

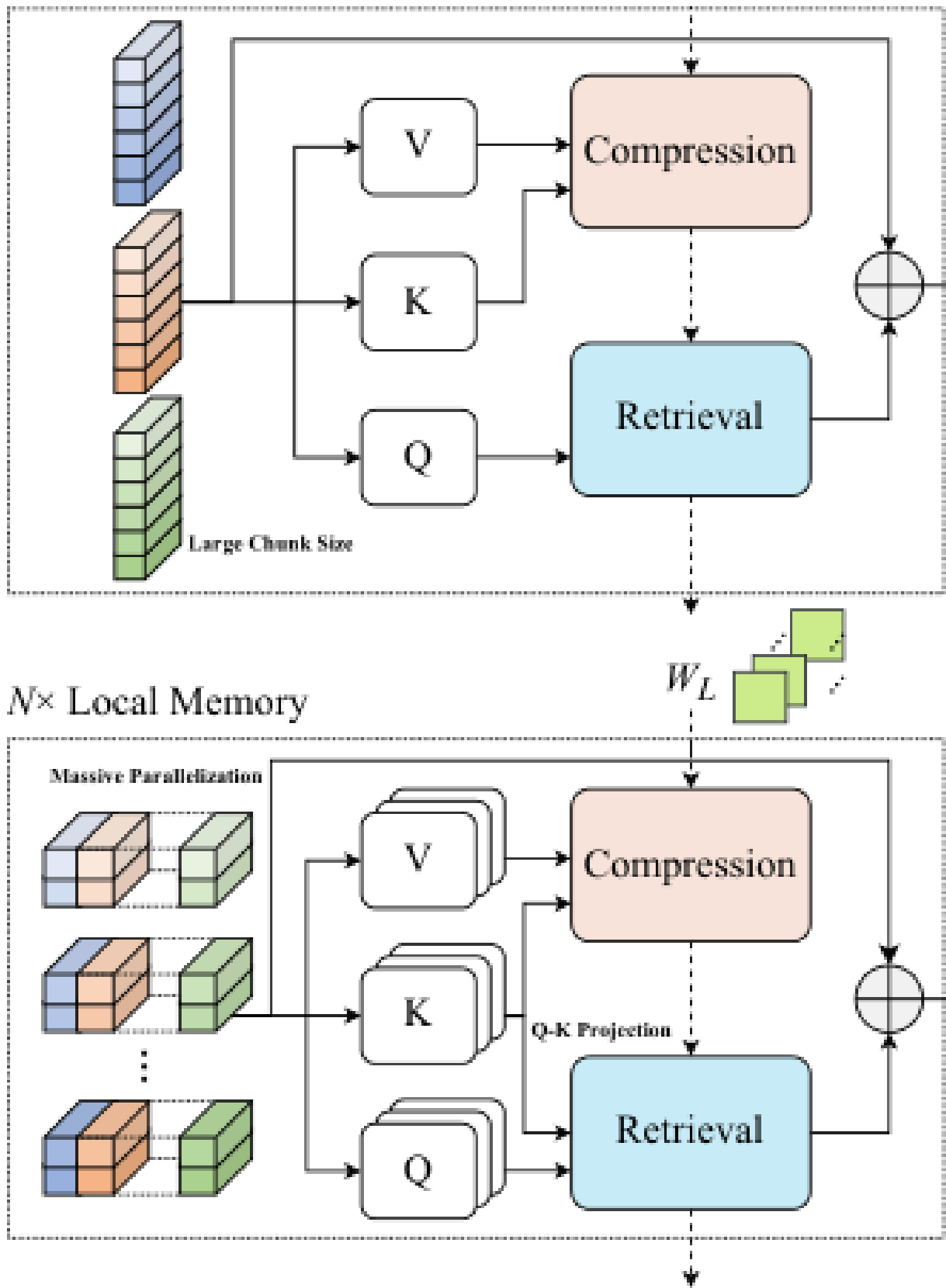


그림 3: TNT 1 단계의 아키텍처 개요. (출처: 원저자, arXiv:2511.07343 Fig.3)

$$V_{(n+1)C_G} \leftarrow V_{nC_G} - \sum_{t=nC_G}^{(n+1)C_G} \eta_t \nabla_V \mathcal{L}(f(V_{nC_G}, k_t), v_t), \quad n = \{0, \dots, L//C_G\} \quad (5)$$

(주: 인쇄본 식 5는 인덱스 n 과 k 를 섞어 표기하는데, 부록 E의 식 13이 단일 청크 인덱스로 일관되게 쓴 형태다. 여기서는 청크 인덱스 하나로 읽으면 된다.)

Local Memory 갱신. local 메모리 W 는 길이 S_L 의 시퀀스 shard 위에서 병렬로 동작한다. 각 shard 안에서 갱신은 더 작은 청크 크기 C_L 을 쓴다.

$$W_t \leftarrow \begin{cases} W_{init} & \text{if } 0 \equiv t \pmod{S_L} \\ W_{t-1} - \sum_{\tau=\xi(t, C_L)}^t \eta_\tau \nabla_W \mathcal{L}(f(W_{\xi(t, C_L)}, k_\tau), v_\tau) & \text{Otherwise} \end{cases} \quad (6)$$

(주: 식 6은 기저 상태를 W_{t-1} 로 인쇄하지만 gradient 의 기준점은 $W_{\xi(t, C_L)}$ 이다. 식 3의 chunkwise 의미—청크 시작상태에서 그 지점을 기준으로 누적한 gradient 합을 뺀 것—와 동일하게 읽는것이 구현 설명과 일관된다.)

global 메모리 갱신 (식 5) 은, 큰 청크들 사이로 상태가 순차적으로 넘겨지는 표준 chunkwise 정식화를 따른다. 훈련 처리량을 극대화하기 위해, global 청크 내 모든 토큰의 gradient 는 그 청크의 초기 상태에 대해 계산되어고도로 병렬화된 갱신을 가능케 한다.

이와 대조적으로 local 메모리 갱신 (식 6) 은 우리의 핵심 혁신인 **주기적 상태 리셋** 을 도입한다. 이 규칙은 local 메모리 상태 W_t 를 길이 S_L 의 각 구간 시작마다 공유·학습가능한 초기 상태 W_{init} 로 리셋하도록 강제한다. 이 주기적 리셋이야말로 입력 전반의 장거리 순차 의존성을 끊어, 세밀한 local 메모리 모듈에 대해 진정한 context 병렬화를 가능케 하는 결정적 메커니즘이다.

이 계층적 설계는 두 갈래로 훈련 효율을 끌어올린다. global 모듈은 큰 청크를 처리해 하드웨어를 포화시키는, 계산 집약적 연산을 만든다. 동시에 local 메모리의 리셋 메커니즘은 context 병렬화를 가능케 하여, 시퀀스를 독립적 청크들로 처리해 여러 장치에 분산하거나 단일 가속기에 쌓아 훈련 처리량을 크게 높인다.

4.1.2 TNT Memory Retrieval: Q-K Projection Challenge 2에서 밝혔듯, 메모리 compression 단계 (식 6) 는 $f(W, \cdot)$ 가 key 공간을 value 공간으로 사상하도록 최적화한다. 그러나 retrieval 시에는 query 벡터 q_t 로 네트워크를 질의하는데, 이는 학습된 key domain 바깥에 놓일 수 있어 성능을 떨어뜨린다.

이를 해결하기 위해 우리는 **Q-K Projection** 을 제안한다. 즉, query q_t 를 이전에 관측된 key 들이 span 하는 부분공간으로 사영한다. 이렇게하면 메모리 함수의 입력이 메모리가 훈련된 공간 안에 놓이게 된다. 최종출력은 global 메모리 (raw query) 와 local 메모리 (사영된 query) 로부터의 retrieval 을 결합한다. 사영은 **local 에만** 적용하는데, local 의 세밀한 성질이 이 mismatch 에 더 민감하기 때문이다.

TNT Memory Retrieval 규칙. 계층적 메모리는 다음과 같이 retrieval 된다.

$$o_t = f(V_{\xi(t, C_G)}, q_t) + f\left(W_t, \sum_{\tau=\xi(t, C_L)}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} q_t\right) \quad (7)$$

이 식은 global 메모리는 raw query 를 그대로 읽고, local 메모리는 관측된 key 부분공간으로 사영된 query 를 읽어 두 결과를 더한다는 뜻이다. 결정적으로, 이 Q-K Projection 은 과거 key 를 모두 저장할 필요가 없다 (그것은 계산 · 메모리상 감당 불가능하다). 대신 사영 행렬 $\sum_{\tau=1}^t k_{\tau} k_{\tau}^{\top} / \|k_{\tau}\|^2 \in \mathbb{R}^{d \times d}$ 를 **running sum(누적 합)** 으로 유지할 수 있다. 그 결과 상태크기가 일정하며 chunkwise 병렬 방식으로 효율적으로 갱신된다. 많은 현대 deep memory module 이 query(q_t) 와 key(k_t) 를 L2 노름으로 정규화하므로, Q-K projection 의 분모가 사라져 $\sum_{\tau=1}^t k_{\tau} k_{\tau}^{\top}$ 로 단순화될 수 있다. 이 효율적 구현의 상세는 부록 C 에 있다.

4.2 TNT 2 단계: 더 미세한 해상도에서의 성능 중심 Fine-tuning

1 단계에서 훈련 효율을 다루었으니, 이제 추론 성능 최적화로 넘어간다. 모델 해상도를 높이는 직관적 방법은 사전학습된 모델을 더 작은 체크 크기로 평가하는 것이다. 그러나 Challenge 3 에서 확인했듯, 사전학습과 평가 체크 크기의 직접적 mismatch 는 상당한 성능 저하를 낳는다.

우리의 핵심 통찰은 이 train-test 괴리가 **최소한의 계산 오버헤드로 교정될 수 있다** 는 것이다. 우리는 사전학습된 모델을 더 작은 local 메모리 체크 크기로 소수의 스텝만 갱신하는 짧은 fine-tuning 단계가, 원래 성능을 회복할 뿐 아니라 흔히 **능가 함** 을 실증적으로 관찰한다.

이 발견에 근거해 우리는 TNT 프레임워크의 2 단계, 곧 성능 중심 fine-tuning 단계를 도입한다. 이 단계에서 우리는 효율적으로 사전학습된 모델을 더 작은 local 체크 크기 ($C'_L < C_L$) 로 계속 훈련한다. 이 과정은 사전학습비용의 극히 일부만으로 모델을 최적 추론에 필요한 고해상도로 적응시킨다. 그렇게 함으로써 2 단계는 Challenge 3 를 직접 해결하여, 효율적 훈련에 필요한 큰 체크 크기와 추론에서 최고 성능을 내는 작은 체크 크기 사이의 간극을 잇는다.

이 2 단계 과정은 사전학습 효율을 추론 요구와 분리한다. 훈련의 대부분은 최대 처리량 (큰 체크) 으로 이루어지고, 최종 모델은 최소한의 오버헤드로 산출된다. 나아가 fine-tuning 은 이상적 추론 시나리오, 즉 **local 체크 크기 1** ($C'_L = 1$) 을 위해 모델을 특화한다. 이는 자기회귀 생성의 표준 **prefill-and-decode** 패러다임과 맞물린다. global 메모리가 문맥의 prefill 을 담당하고, 최적화된 local 메모리가 반복적 decode 를 담당한다.

5 실험 (Experiments)

우리는 2 단계 훈련 프레임워크 TNT 를 실증적으로 평가한다. TNT 는 모델에 구애받지 않지만 (model-agnostic), 그 효과를 보이기 위해 강력한 deep memory 모델인 Titans (Behrouz, Zhong, et al. 2025) 로 인스턴스화한다. 실험에서 훈련 시간과 모델 정확도에 관한 주장을 검증한다.

5.1 실험 설정

Baseline. 우리는 여러 강력한 장문 아키텍처와 비교한다. 주된 비교 대상은 base 모델인 Titans (Behrouz, Zhong, et al. 2025) 이다. 또한 vanilla Transformer (Vaswani et al. 2017), Gated Transformer (Qiu et al. 2025), TTT (Sun, Li, et al. 2024) 와도 비교한다.

훈련 및 TNT 설정. 우리는 (Behrouz, Zhong, et al. 2025) 를 따라 150M 파라미터 모델을 훈련하며, T5 tokenizer (32k vocab) 를 쓴다. 최적화기는 AdamW (Loshchilov et al. 2017), weight decay 0.1, cosine 스케줄 (peak LR 1×10^{-3}) 이다. 실험은 TPUv5 pod (2×2×2 토폴로지, model parallelism 2) 에서 수행한다. TNT 의 경우 N 개 local 모듈 설정을 그 체크 크기들의 집합 $C_L = \{C_{L,1}, \dots, C_{L,N}\}$ 로 표기한다. 예를 들어 $C_L = \{8, 16\}$ 은 체크 크기 8 과 16 을 갖는 두개의 local 모듈을 뜻한다. global 메모리는 $C_G = 2048$ 을 쓴다.

실험 구성. 효율 벤치마크 (5.2 절) 에서는 문맥 길이를 2k – 32k 로 변화시키고, 배치당 0.5M 토큰, local window $S_L = 2048$ 을 쓴다. 성능평가 (5.3 절) 에서는 문맥 길이 16k, 배치당 1M 토큰, $S_L = 4096$ 을 쓴다.

TNT	JAX	{8}	2.54	7.68×
TNT	JAX	{16}	1.65	11.78×
TNT	JAX	{32}	1.22	15.92×
TNT	JAX	{64}	1.12	17.37×
TNT	JAX	{128}	1.16	16.75×

quences. As sequence length increases, TNT’s superior scalability creates a crossover point where it becomes faster. This efficiency is most pronounced at very long contexts; for instance, at a sequence length of 32K, a implementation of TNT ($C_L = 128$) outperforms even the highly optimized FlashAttention kernel, confirming y for demanding long-context training scenarios.

Quality Comparison. We next translate step runtime gains into a practical time-to-quality. As shown in Table 1, our TNT framework accelerates the total training time required for target model quality. Our best configuration is up to **17.4×** faster than the original Titans. This efficiency gain is fundamental to the ; for instance, using an identical local memze of 8, TNT is already **7.7×** faster than its terpart. While competitive with standard sformers in JAX, our implementation does perform highly optimized baselines like the sformer with FlashAttention (Dao, Fu, et al. s an expected result, as TNT currently lacks a el, which we leave for future work. Nonetheless, these results establish TNT as an efficient founda-

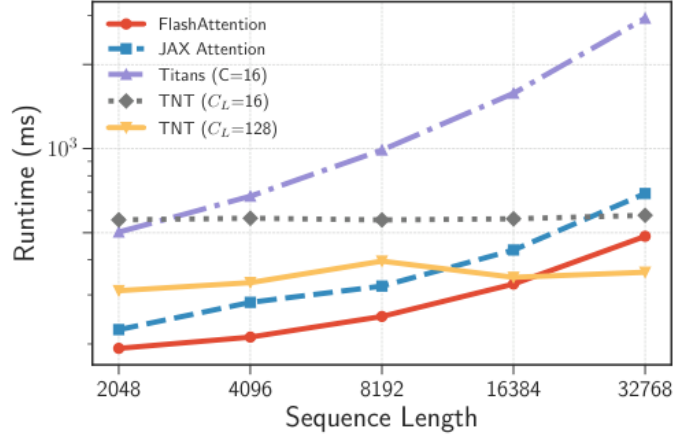


그림 4: 서로 다른 모델과 구현의 시퀀스 길이별 런타임 비교. 배치당 토큰 수는 0.5M 으로 고정. 추가 결과는 Figure 5 에 있다. (출처: 원저자, arXiv:2511.07343 Fig.4)

5.2 TNT 를 통한 더 빠른 Memory 훈련

시퀀스 길이에 대한 선형 런타임 확장. 우리는 먼저 배치당 토큰수를 고정한 채 시퀀스 길이를 바꿔 단일 스텝 런타임 성능을 분석한다. Figure 4 에서 보듯, TNT 의 런타임은 시퀀스 길이에 대해 선형으로증가하는 반면, Titans 와 표준 attention 은 이차로 증가한다. 이 확장 이점은 긴 문맥에서 두 드러진다. 32K 시퀀스 길이에서 TNT 는 동일한 메모리 청크크기 ($C_L = C = 16$) 를 갖는 Titans 보다 **5.1 배** 빠르다. 또한 local 청크 크기가 클수록 TNT 의 속도가 일관되게 향상되어, $C_L = \{128\}$ 일 때 TNT 는 고도로 최적화된 FlashAttention 커널 (Dao 2024) 보다도 **1.3 배** 빠르다.

TNT 의 고도로 병렬화 가능한 아키텍처는 시퀀스 길이에 선형으로 확장되는런타임을 달성하며, 이는 표준 attention 의 이차 복잡도 대비 핵심 이점이다. Titans 같은 모델도 이론상 선형이지만, 내재된 순차 의존성이 효과적병렬화를 막아 하드웨어 활용률이 낮고 긴 시퀀스에서 wall-clock 이 느리다. 시퀀스 길이가 늘수록 TNT 의 우수한 확장성은 교차점 (crossover) 을 만들어훨씬 빨라진다. 이 효율은 매우 긴 문맥에서 가장 두드러진다. 예컨대 32K 길이에서 순수 JAX 구현의 TNT($C_L = 128$) 가 고도로 최적화된 FlashAttention 커널마저 능가한다.

Time-to-Quality 비교. 다음으로 이 단일 스텝 런타임 이득을실용적인 time-to-quality 관점으로 옮긴다. Table 1 에서 보듯, TNT 프레임워크는 목표 모델 품질에 도달하는 데 필요한 총 훈련 시간을 크게단축한다. 우리의 최선 구성은 원래 Titans baseline 대비 최대 **17.4 배** 빠르게 이를 달성한다. 이 효율 이득은 아키텍처에 내재된것이다. 예컨대 동일한 local 메모리 청크 크기 8 을 쓸 때, TNT 는 이미 Titans 대응 모델보다 **7.7 배** 빠르다. JAX 의 vanilla Transformer 와는 경쟁력이 있으나, FlashAttention 을 쓴 Gated Transformer 같은 고도 최적화 baseline 은 아직 능가하지 못한다. 이는 예상된 결과인데, TNT 에는 아직 맞춤 커널이 없기 때문이며 이는 후속 연구로 남긴다. 그럼에도이 결과들은 TNT 가 recurrent 모델 연구를 위한 효율적 토대이며, SOTA Transformer 의 속도에 이르는 명확한 경로를 갖고 있음을 확립한다.

Table 1: TNT 는 목표 훈련 손실에 baseline Titans 대비 최대 17 배빠르게 도달한다. 서로

다른 150M 모델이 동일 목표 손실 3.20 에 도달하는 데 걸린 시간을 비교한다.

Model	Implementation	C 또는 C_L	Training Time (hrs)	Speedup
Titans	JAX	8	19.48	$1.00\times$
Titans	JAX	16	10.79	$1.81\times$
Titans	JAX	32	6.45	$3.02\times$
Titans	JAX	64	4.18	$4.67\times$
Titans	JAX	128	3.71	$5.25\times$
Transformer (w/o gating)	JAX	-	1.74	$11.18\times$
Transformer (w gating)	JAX	-	1.38	$14.10\times$
Transformer (w/o gating)	FlashAttention (Pallas)	-	1.23	$15.90\times$
Transformer (w gating)	FlashAttention (Pallas)	-	0.96	$20.22\times$
TNT	JAX	{8}	2.54	$7.68\times$
TNT	JAX	{16}	1.65	$11.78\times$
TNT	JAX	{32}	1.22	$15.92\times$
TNT	JAX	{64}	1.12	$17.37\times$
TNT	JAX	{128}	1.16	$16.75\times$

5.3 TNT 는 모델 품질을 향상시킨다

우리의 TNT 프레임워크는 모델 품질을 크게 향상시켜, 강력한 RNN 기반 baseline 과 표준 Transformer 구현을 능가한다. Table 2 에서 보듯, 초기 1 단계 사전학습만으로도 매우 효과적이다. 우리의 최선 1 단계 모델은 평균 perplexity **23.13** 을 달성하는데, 이는 최고 성능 Titans(25.07) 와 vanilla Transformer(23.58) 에 비해 뚜렷한 개선이다. TNT 가 SOTA 인 Gated Transformer 의 perplexity(22.39) 를 완전히 따라잡지는 못하지만, 상식 추론과제에서 더 높은 평균 정확도 (41.0% 대 39.7%) 를 달성한다. 이 규모에서는 하위 과제 정확도의 분산이 클 수 있으므로, 우리는 perplexity 를 언어모델링 능력의 더 안정적인 지표로 본다.

나아가 2 단계 fine-tuning 은 성능을 추가로 끌어올리는 효율적 방법을 제공한다. 이 단계는 계산량 저렴하여 원래 사전학습 계산의 약 5% 만 더 든다 (Table 4 참조). 그럼에도 평균 perplexity 를 최종 **23.09** 로 일관되게 낮춘다. 이 결과들은 TNT 가 기존 RNN 기반 아키텍처의 한계를 넘어서는 고품질 모델을 산출하는 효과적 프레임워크이며 표준 Transformer 의 강력한 대안임을 입증한다.

Table 2: TNT(150M 파라미터) 와 baseline 들의 언어모델링 · 상식 추론 과제 성능 (각기 10B 토큰으로 훈련). TNT 모델은 global 청크 크기를 $C_G = 2048$ 로 고정. C_L 은 local 메모리 모듈이 쓰는 청크 크기 집합.

Model	C 또는 C_L	C4 ppl ↓	FineWebPG19 ppl ↓	pgl ↓	Avg. ppl ↓	PIQA acc ↑	Hella. acc ↑	ARC-e acc ↑	CSQA acc ↑	Avg. acc ↑
Trans- former (w/o gat- ing)	-	20.98	20.59	29.18	23.58	62.0	30.9	34.8	25.5	38.3
Trans- former (w gat- ing)	-	19.82	19.61	27.75	22.39	63.3	32.2	36.8	26.7	39.7

Model	C 또는 C_L	C4 ppl ↓	FineWeb ppl ↓	WebPG19 ppl ↓	Avg. ppl ↓	PIQA acc ↑	Hella. acc ↑	ARC-e acc ↑	CSQA acc ↑	Avg. acc ↑
TTT (2024)	256	24.18	24.31	34.36	27.62	60.6	30.8	34.1	26.9	38.1
Titans (2025)	256	23.53	24.13	33.73	27.13	61.3	30.8	35.1	27.8	38.8
Titans	8	22.25	22.07	30.90	25.07	60.8	32.0	35.5	27.8	39.0
TNT	{8}	21.04	21.01	30.24	24.10	61.8	32.8	37.4	30.3	40.6
Stage										
1										
TNT	{8,16}	20.74	20.73	29.94	23.80	63.5	32.4	37.4	30.6	41.0
Stage										
1										
TNT	{4,8,16}	20.47	20.43	29.43	23.44	62.9	32.4	36.4	28.9	40.2
Stage										
1										
TNT	{4,8,16,32}	20.15	20.17	29.08	23.13	63.2	32.0	36.7	30.3	40.6
Stage										
1										
TNT	{1}	20.86	20.91	30.21	23.99	63.2	32.8	37.4	30.1	40.9
Stage										
2										
TNT	{2,4}	20.65	20.70	29.97	23.77	63.4	32.5	37.3	30.2	40.9
Stage										
2										
TNT	{2,4,8}	20.32	20.35	29.42	23.36	64.0	32.0	36.9	28.1	40.3
Stage										
2										
TNT	{2,4,8,16}	20.10	20.13	29.05	23.09	63.5	32.3	37.4	30.2	40.9
Stage										
2										

5.4 Ablation 연구

우리는 TNT의 핵심 설계 선택을 검증하기 위해 ablation 연구를 수행했고, 결과를 Table 3에 정리했다.

계층적 메모리. 계층적 설계의 효과는 분명하다. local 메모리모듈을 점진적으로 추가하면 Titans baseline 대비 성능이 일관되게 개선되어, local 모듈 4개일 때 perplexity가 23.53에서 20.15로 낮아진다. 반대로 global 메모리를 제거하면 성능이 악화된다 (PPL이 25.60으로 증가). 이는 local 메모리의 리셋메커니즘 때문에 소실되는 장거리 의존성을 포착하는 데 global 메모리가 결정적 역할을 함을 확인해 준다.

Q-K Projection. query-key 사영은 성능에 필수적임이 드러난다. 이를 제거하면 상당한 손해 (PPL이 21.04에서 22.01로 증가)가 발생하여, compression-retrieval mismatch (Challenge 2)를 완화하기 위해 필요하다는 우리의 가설을 뒷받침한다.

2단계 Fine-tuning. 2단계 fine-tuning을 적용하면 언어모델링 (20.86 PPL)과 상식 추론 (40.9%)이 모두 향상되어, 사전학습된 모델을 고해상도 추론에 맞춰 적응시키는 데 효과적임을 보여준다.

Table 3: TNT ablation 연구. 제안된 각 변경이 deep memory module에 기여하는 정도를 보여준다.

TNT	N	Language Modeling ppl ↓	C.S. Reasoning acc ↑
Base Model (Titans)	-	23.53	38.8
+1 Local Memory	1	21.04	40.6
+2 Local Memory	2	20.74	41.0

TNT	N	Language Modeling ppl ↓	C.S. Reasoning acc ↑
+3 Local Memory	3	20.47	40.2
+4 Local Memory	4	20.15	40.6
w/o global memory	1	25.60	35.5
w/o Q-K projection	1	22.01	36.4
w Stage 2	1	20.86	40.9

(각주: 본 연구는 TTT/Titans 모델을 저자들이 직접재구현한 것에 기반한다. 공정한 비교를 위해 모든 모델을 동일 설정에서 훈련했다.)

6 결론 (Conclusion)

우리는 deep memory module 에서 훈련 효율과 추론 성능 사이의 근본적 상충을 해소하는 2 단계 훈련 프레임워크 TNT 를 도입했다. 주기적 상태리셋을 갖춘 계층적 메모리 아키텍처를 활용하여, TNT 는 사전학습 중 대규모 context 병렬화를 가능케 하고, 이어서 고해상도 추론을 위한 효율적 fine-tuning 을 수행한다. 우리의 실험은 가장 정확한 RNN baseline 대비 최대 **17 배** 가속을 달성하면서 동시에 성능까지 향상시킴을 보여준다. TNT 는 중대한 확장성 병목을 제거하여 deep memory module 의 실용성을 크게 개선하고, Transformer 와의 성능 격차를 좁히는 후속 연구를 촉진한다.

부록 A: 관련 연구 (Related Work)

현대 Linear Recurrent Neural Network. Transformer 의 이차 복잡도 때문에 최근 대안 아키텍처 개발이 주목받았고, 이는 효율적 recurrent 대안의 발전으로 이어졌다 (Tiezzi et al. 2024). 이 분야의 초기 진전은 RetNet(Sun, Dong, et al. 2023), RWKV(Peng, Alcaide, et al. 2023), S5(Smith et al. 2023) 같은 모델로 시작되며, 이들은 데이터 독립적 전이행렬과 Hebbian 유사 갱신 메커니즘을 결합했다. 이어 입력 의존적 파라미터를 이 선형 아키텍처에 도입한 2 세대 모델들 (예: linear RNN(Hasani et al. 2023; Smith et al. 2023), RWKV6(Peng, Goldstein, et al. 2024)) 이 등장했다. 이들은 특히 delta rule 에 기반한 더 표현력 있는 메모리 갱신 규칙도 탐구했다 (Peng, Zhang 등; Schlag et al. 2021; Yang 등; Liu et al. 2024). 이 계보의 후속 진화는 이 메모리 아키텍처를 더 깊은 모델로 확장하면서, delta-rule 유사 갱신 (Sun, Li, et al. 2024) 이나 forget gating 을 갖춘 데이터 의존적 momentum 기반 갱신 (Behrouz, Zhong, et al. 2025) 을 함께 활용했다. 더 최근에는 delta-rule 기반 순차 모델의 성능을 높이기 위해 Siems et al.(2025) 이 토큰당 여러 번의 gradient descent 갱신을 제안하여, 특히 state tracking 과제에서 더 표현력 있는 모델을 얻었다. 이 밖에도 여러 연구가 비선형 recurrence 를 갖는 RNN 과 그 훈련을 더 빠르게 하는 방법에 초점을 두었다 (Behrouz 등; Csordás et al. 2024; Merrill et al. 2024; Lim et al. 2024; Schöne et al. 2025; Karami and Mirrokni 2025; Gonzalez et al. 2024).

Fast Weight Program. 선형 층을 key-value associative memory 로 보는 개념화는 Hopfield network(Hopfield 1982) 로 거슬러 올라간다. 이 개념은 이후 fast weight programmer 의 맥락에서 발전했는데, 여기서 동적 fast program 이 recurrent 신경망에 통합되어 쓰기 가능한 메모리 저장소로 기능한다 (Schlag et al. 2021; Schmidhuber 1992; 1993). 이런 시스템의 학습 패러다임 가운데 Hebbian 학습 (Hebb 2005) 과 delta rule(Prados et al. 1989) 이 가장 두드러졌으며, 두 규칙 모두 기존 문헌에서 폭넓게 연구되어 왔다.

Hopfield Network. 우리의 정식화는 key 와 value 사이의 사상을 학습하는 것을 주목적으로 하는 associative memory 라는 넓은 개념 위에 아키텍처적으로 세워져 있다. Hopfield(1982) 의 선구적 연구는 associative memory 에 명시적으로 기반한 초기 신경 아키텍처 중 하나를 도입하여, key-value 쌍을 저장하기 위한 에너지 함수의 최소화를 이를 정의했다. 전통적 Hopfield network 는 벡터 값 메모리 용량의 제약과 에너지 함수의 성질 때문에 근래 활용이 줄었으나, 여러 현대 연

구가 다양한 방법으로 그 용량 향상에 초점을 두었다 (Krotov 2021; Li et al. 2024; Krotov and Hopfield 2016). 특히 지수 커널을 결합한 에너지 함수 확장이 탐구되었고 (Krotov and Hopfield 2016; Lucibello et al. 2024), 이렇게 현대화된 Hopfield network와 Transformer 아키텍처 사이의 관계도 최근 연구주제가 되었다 (Ramsauer et al. 2021; Hu, Wu, et al. 2024).

부록 B: Q-K Projection 과 Memory Bounded Transformer 의 연결

우리의 QK-Projection retrieval 과정을 다시 보면, 먼저 query 벡터를 저장된 key 들의 공간으로 사영한 뒤, 메모리 모듈 위에서의 forward pass 로 대응하는 value 를 retrieval 한다. 구체적으로 query q_t 와 저장된 key $\{k_i\}_{i=1}^t$ 가 주어질 때, query q_t 에 대응하는 출력은 다음과 같이 계산된다.

$$o_t = f\left(W_t, \sum_{\tau=1}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} q_t\right) \quad (8)$$

key 정규화 ($\|k_\tau\|^2 = 1$) 를 가정하면 이 식은 다음과 같이 다시 쓸 수 있다.

$$o_t = f\left(W_t, \sum_{\tau=1}^t k_\tau k_\tau^\top q_t\right) \quad (9)$$

여기서 두 번째 요소 $\sum_{\tau=1}^t k_\tau k_\tau^\top q_t$ 는, $\mathcal{M}'_t = \sum_{\tau=1}^t k_\tau k_\tau^\top$ 인 linear memory module 위에서 query q_t 에 대한 단순 forward pass 와 동치이며, 그 recurrence 는 다음과 같다.

$$\mathcal{M}'_t = \mathcal{M}'_{t-1} + k_t k_t^\top \quad (10)$$

이런 QK-Projection 의 정식화는 memory bounded Transformer (Peng, Kasai 등 2022; Zhang, Yang 등 2024; Karami, Behrouz 등 2025) 의 **two-pass** 과정을 떠올리게 한다. 단순 linear attention 형태 (Peng, Kasai 등 2022) 에서 retrieval 과정은 다음과 같이 쓸 수 있다.

$$W_t = W_{t-1} + \varphi_t v_t^\top \quad (11)$$

$$o_t = W_t \text{softmax} \left(\sum_{\tau=1}^t k_\varphi \varphi_\tau^\top q_t \right) \quad (12)$$

위의 ABC(Peng, Kasai 등 2022)의 two-pass 과정과 비교하면, 우리의 QK-projection 방법은 **deep** 및 **linear** 메모리 양쪽에 모두 적용가능 하다. 또한 φ_t 의 파라미터가 k_t 와 묶여 (tied) 있어 학습 과정이 상당히 쉬워지고, 새로운 데이터/과제에 더 잘 적응한다. 나아가 이 사영을 local 메모리에 적용할 때는 합을 $\tau = 1$ 이 아니라 메모리의 “리셋” 상태에서부터만 취한다.

부록 C: Q-K Projection의 효율적구현

이 절은 QK-Projection 메커니즘의 효율적·병렬화 가능한구현을 상술한다. 우리는 이 사영이 순차적 병목을 도입하지 않고 chunkwise 훈련 패러다임에 통합될 수 있어, TNT 아키텍처의 훈련 효율을 보존함을 보인다.

QK-Projection은 현재 local 메모리 shard(길이 S_L) 내 key들의 외적을누적하는 사영 행렬 $\mathcal{M}_t$ 에 의존한다. 정규화된 key($\|k_\tau\| = 1$)를 가정하면, 이 행렬은 다음 recurrence로 정의된다.

$$\mathcal{M}_t = \begin{cases} k_t k_t^\top & \text{if } t \equiv 1 \pmod{S_L} \\ \mathcal{M}_{t-1} + k_t k_t^\top & \text{otherwise} \end{cases}$$

이 규칙은 사영 상태 $\mathcal{M}_t$ 가 각 shard 시작에서 리셋되도록하여, local 메모리 상태 W_t 의 리셋을 그대로 반영한다. 그러면 local 메모리 retrieval은 $f(W_t, \mathcal{M}_t q_t)$ 로 계산된다.

Chunkwise 병렬 계산. 훈련 효율을 유지하기 위해 $\mathcal{M}_t$ 는 chunk-parallel 방식으로 계산된다. 크기 C_L 의 청크 내 임의의 시점 t 에 대해, 사영 행렬은 두 성분으로 분해된다.

$$\mathcal{M}_t = \underbrace{\mathcal{M}_{\xi(t, C_L)-1}}_{\text{Carry-over State}} + \underbrace{\sum_{\tau=\xi(t, C_L)}^t k_\tau k_\tau^\top}_{\text{Intra-chunk Sum}}$$

첫 항은 이전 청크에서 넘겨받은 최종 상태다. 둘째 항인 청크 내 합은, 외적수열 $\{k_\tau k_\tau^\top\}$ 에 대한 병렬 prefix sum(scan) 연산으로 청크 내 모든 스텝에 대해 동시에 효율적으로 계산된다.

이 구현은 중단 간 병렬성을 보존한다. 청크 사이로 넘겨지는 상태는 단일 고정 크기 행렬 ($d \times d$)이라 오버헤드가 미미하다. 주기적 리셋은 shard 경계에서 이 carry-over 상태를 재초기화함으로써 처리된다. 따라서 QK-Projection은 TNT 아키텍처에 근본적인 훈련 효율을 저해하지 않으면서 모델의 retrieval 메커니즘을 강화한다.

부록 D: TNT의 적용 가능성 (TNT Applicability)

본 논문에서 우리는 TNT의 효과와 효율을 보이는 데 초점을 두었고, 명료함을 위해 inner-loop를 gradient descent로 최적화하는 단순한 메모리 모듈을 사용했다. 그러나 TNT 레시퍼는 서로 다른 deep memory 및 비선형 아키텍처에 적용 가능하다. 예를 들어 Titans(Behrouz, Zhong, et al. 2025)나 Mamba2(Dao and Gu 2024)의 gating 정식화를 각 local 메모리와 global 메모리에 적용할 수 있다. 또 다른 탐색은 Hu, Pan, et al.(2025)처럼 inner-loop의 목적에 closed feedback loop를 통합하는 것이다. 마찬가지로 inner-loop 최적화기로 momentum을 갖는 gradient descent, AdamW(Kingma et al. 2014), muon(Jordan et al. 2024) 같은 더 표현력 있는 최적화기를 쓸 수도 있다 (Behrouz, Li, et al. 2025; Zhang, Bi, et al. 2025에서 수행된 바 있다). 이 모든 조합을 TNT와 탐구하는 것은 유망한 방향이지만, 명료함과 지면 제약을 위해 후속 연구로 남긴다.

부록 E: TNT 일반화 정식화 (Generalization Formulation)

TNT 아키텍처는 하나의 global 메모리와 N 개의 병렬 local 메모리로 이루어진 계층 시스템으로 일반화될 수 있다. 이는 모델이 여러 시간 척도와 해상도의 정보를 동시에 포착하게 해 준다. 각 local 메모리 $W^{(i)} (i \in \{1, \dots, N\})$ 는 고유한 청크 크기 C_{L_i} , shard 길이 S_{L_i} , 학습 가능한 초기 상태 $W_{init}^{(i)}$ 를 갖고 동작한다.

E.1 일반화된 메모리 갱신

갱신 규칙은 다음과 같이 확장된다. 단일 global 메모리는 순차적으로 진화하고, N 개의 local 메모리는 각자의 독립적 스케줄로 병렬 갱신된다.

Global Memory 갱신. global 메모리 상태 V 는 base 경우와 동일하게 큰 청크 크기 C_G 로 순차적으로 갱신된다.

$$V_{(k+1)C_G} \leftarrow V_{kC_G} - \sum_{t=kC_G}^{(k+1)C_G} \eta_t \nabla_V \mathcal{L}(f(V_{kC_G}, k_t), v_t) \quad (13)$$

N-Local Memory 갱신. N 개의 local 메모리 $W^{(i)}$ 각각이 병렬로 동작한다. 각 메모리의 상태는 자신의 shard 길이 S_{L_i} 에 따라 주기적으로 리셋되어, 다중 해상도 context 병렬화를 가능케 한다.

$$W_t^{(i)} \leftarrow \begin{cases} W_{init}^{(i)} & \text{if } 0 \equiv t \pmod{S_{L_i}} \\ W_{t-1}^{(i)} - \sum_{\tau=\xi(t, C_{L_i})}^t \eta_\tau \nabla_{W^{(i)}} \mathcal{L}(f(W_{\xi(t, C_{L_i})}^{(i)}, k_\tau), v_\tau) & \text{Otherwise} \end{cases} \quad (14)$$

여기서 $i = 1, \dots, N$ 이다.

E.2 일반화된 메모리 Retrieval

retrieval 시 최종 출력은 global 메모리와 N 개 local 메모리 출력의 합성이다. global 메모리는 raw query q_t 를 쓰고, 각 local 메모리는 자신의 청크 크기 C_{L_i} 로 정해지는 특정 문맥 창에 맞춘 Q-K 사영을 쓴다.

TNT 일반화 Retrieval 규칙. 계층적 메모리는 각 메모리 모듈의 기여를 합산해 retrieval 된다.

$$o_t = f(V_{\xi(t, C_G)}, q_t) + \sum_{i=1}^N f\left(W_t^{(i)}, \sum_{\tau=\xi(t, C_{L_i})}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} q_t\right) \quad (15)$$

이 정식화는 네트워크가 global 메모리로부터의 장거리 의존성과, 서로 다른 시간적 패턴에 특화된 다양한 local 메모리들로부터의 세밀하고 병렬 처리된 정보를 통합하게 해 준다.

Table 4: 10B 토큰으로 훈련한 150M 파라미터 모델의 훈련 시간. TNT 모델은 global 청크 크기를 $C_G = 2048$ 로 고정하고, C_L 은 local 메모리 모듈의 청크 크기 집합.

Model	C 또는 C_L	Training Time (hrs)
Transformer (w/o gating)	-	0.80
Transformer (w gating)	-	0.82
TTT (2024)	256	1.69
Titans (2025)	256	1.99
Titans	8	8.44
TNT Stage 1	{8}	3.06
TNT Stage 1	{8,16}	4.24
TNT Stage 1	{4,8,16}	5.00
TNT Stage 1	{4,8,16,32}	5.55
TNT Stage 2	{1}	0.15
TNT Stage 2	{2,4}	0.23
TNT Stage 2	{2,4,8}	0.26
TNT Stage 2	{2,4,8,16}	0.46

Model	C 또는 C_L	Training Time (hrs)
-------	------------	---------------------

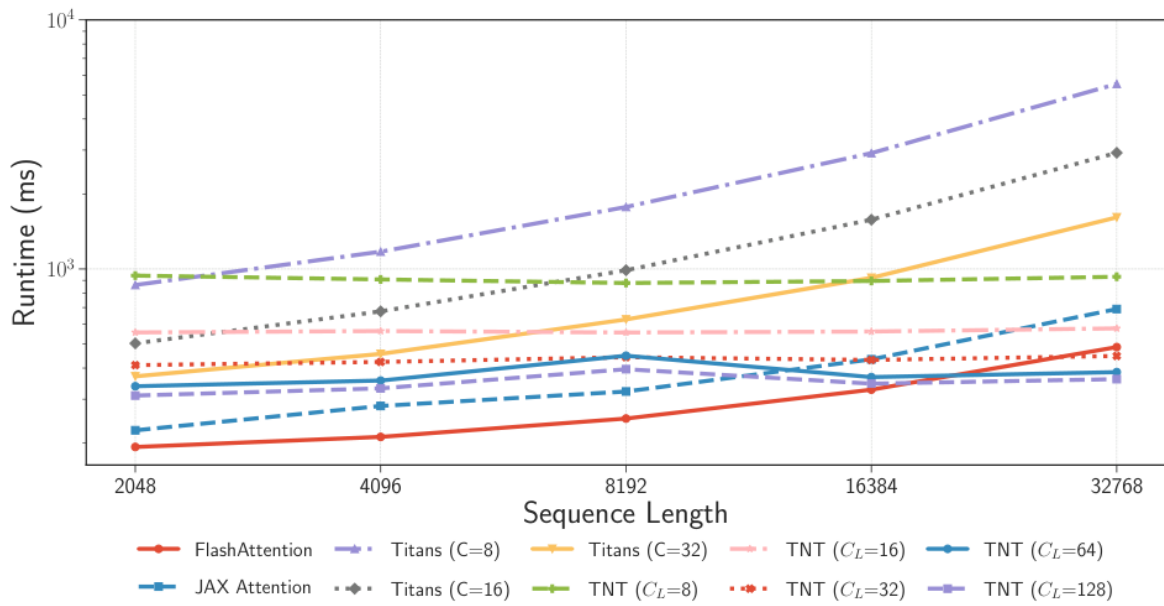


그림 5: 서로 다른 모델과 구현의 시퀀스 길이별 런타임 비교. 배치당 토큰 수는 0.5M 으로 고정. (출처: 원저자, arXiv:2511.07343 Fig.5)